Algoritmos

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros compandos de R

R es un lenguaje muy parecido a Matlab y trabaja generalmente con variables que comumente son matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

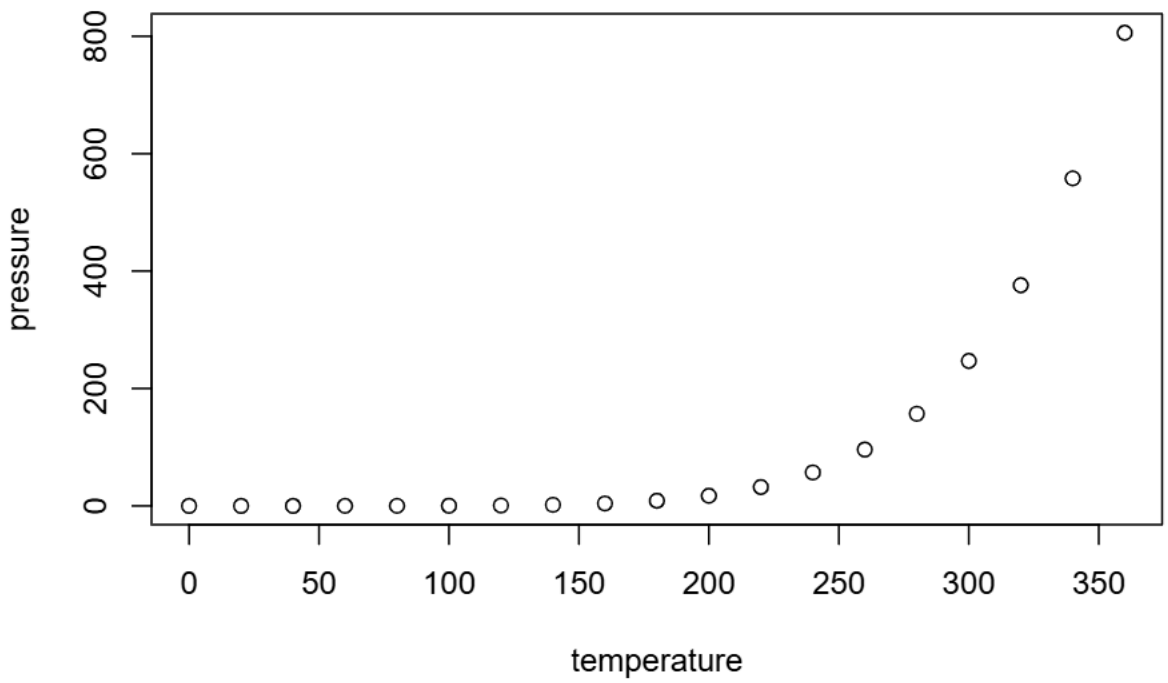
```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

##Uso de datasets o de base de datos interna de R

Usando el compando data en la consola obtenemso distintos datos ya ccargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ despues del comando de información específico ## Including Plots

You can also embed plots, for example:



##Comando Ploot Es un comando que permite graficar cualquier fuente de datos que le asignemos,teniendo tabmbien la posibilidad de agragar titulos, referncias en los ejes y demas herrmanientas, poniendo el comando ?plot, luego help, parecerá todos los comandos posibles qu epodemos utilizar y como utilizarlos Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

##Funciones Estadisticas

```
z1<-rnorm(350,22,5)
z1
```

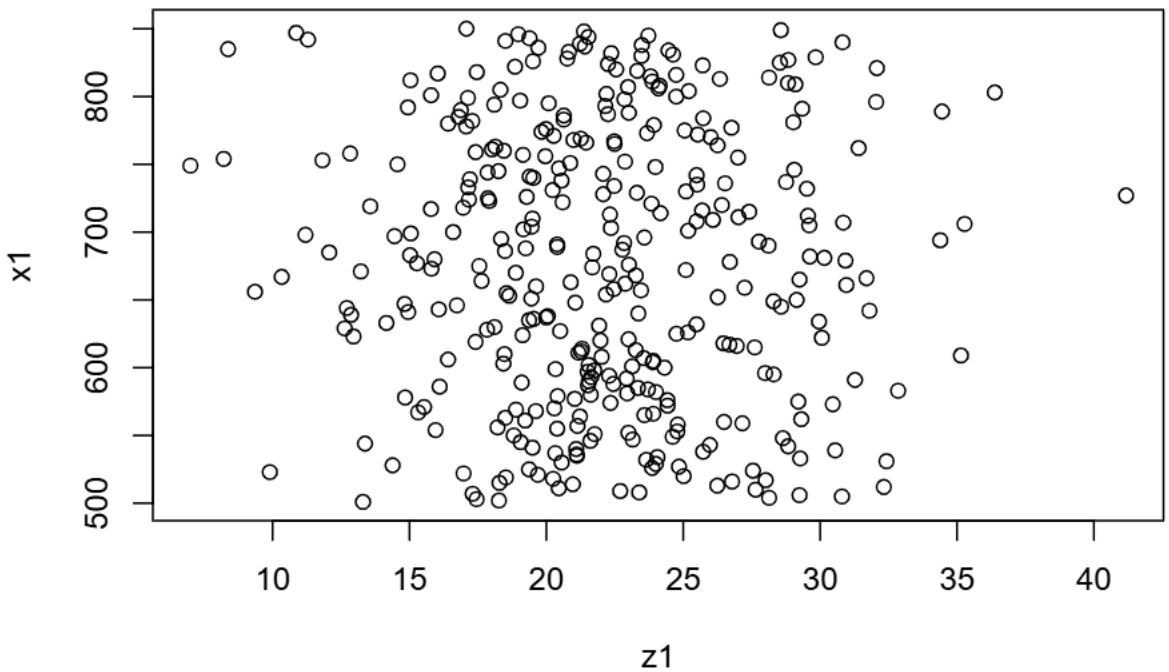
```
##   [1] 13.300757 18.268687 17.449179 28.137345 30.798611 29.247818 17.307871
##   [8] 23.392110 22.697945 27.643397 20.453508 32.328930 26.243252 20.965624
##  [15] 18.286025 26.784765 28.011181 20.245904 18.521747 25.012803 19.690257
##  [22] 16.976741  9.899364 27.546551 19.370377 23.864115 24.842656 14.383549
##  [29] 23.998683 20.561656 32.422975 23.647814 29.273204 24.056462 21.115877
##  [36] 21.105485 20.325533 25.726108 30.550032 21.090838 19.494184 28.834052
##  [43] 25.973489 13.376349 19.063149 21.608511 23.152673 28.641077 24.614661
##  [50] 18.808588 21.764526 22.997571 24.782577 15.953862 20.394287 18.218468
##  [57] 21.135005 24.795944 27.161238 26.488549 19.225551 29.311957 18.500259
##  [64] 21.225488 23.582103 23.901593 15.319569 19.616026 18.889619 20.289738
##  [71] 15.531316 24.427869 30.462643 22.341955 29.203097 24.423659 21.034777
##  [78] 14.839398 20.412260 21.620090 22.953615 24.006232 32.849338 23.708980
##  [85] 23.345845 16.101536 21.524148 22.443454 19.099709 21.553670 31.272819
##  [92] 22.930411 21.636314 22.291901 28.299180 27.990547 21.511643 21.747363
##  [99] 20.333661 24.316927 23.134159 21.542147 18.431567 23.888396 23.907876
## [106] 16.408985 23.559407 22.018934 35.143936 18.470105 21.173437 21.270339
## [113] 23.264679 21.300559 27.610611 26.954867 26.684256 26.470524 17.413942
## [120] 21.973574 23.003130 30.063694 12.955093 19.130296 24.753019 25.175349
## [127] 20.503608 17.831601 12.631599 18.106094 21.925716 25.484506 14.157192
## [134] 29.958769 19.366498 19.539239 19.993732 20.050587 12.865283 23.361842
## [141] 14.952049 31.802608 16.069433 12.709148 28.571161 16.732996 14.822456
## [148] 21.058466 28.297606 29.143673 19.453732 26.262427 18.640279 22.179224
## [155] 18.547969  9.357995 23.454176 22.462463 27.243142 19.622479 30.959564
```

```
## [162] 22.885832 20.888850 17.632882 29.248838 31.695098 10.332118 23.264665
## [169] 22.298399 18.885200 13.222571 25.095412 15.802278 21.676959 17.547599
## [176] 23.014295 15.272516 26.700529 30.922672 15.916782 30.153762 29.624531
## [183] 15.022465 21.713864 12.070139 18.489141 22.776448 19.247792 20.399186
## [190] 28.114739 20.392696 22.840003 27.765752 34.388254 18.346397 23.580306
## [197] 14.453610 11.202036 15.047443 16.593243 25.180032 19.159979 22.357830
## [204] 19.462280 29.602529 35.276384 30.846780 25.490980 26.084294 19.493269
## [211] 27.018691 29.553468 22.322666 24.170336 27.407496 25.692146 15.784543
## [218] 16.963150 13.568052 26.409405 23.840167 20.594263 17.908833 17.168804
## [225] 17.868788 19.281752 41.175845 22.075683 23.306968 25.102597 20.228055
## [232] 29.517166 17.148551 22.477297 25.507209 26.528736 28.757467 20.553682
## [239] 17.204189 19.524114 19.394635 25.487207 22.073833 17.855075 18.250210
## [246] 29.055310 20.460607 23.984892  6.997621 14.560894 20.869543 22.872329
## [253] 11.821229  8.213553 27.000322 19.962219 19.146592 12.834258 17.417160
## [260] 18.445604 18.014232 31.405171 18.137787 26.253485 22.493479 21.451877
## [267] 22.478203 20.992062 21.257506 26.004140 20.266657 25.532383 23.676269
## [274] 19.823145 25.051418 19.988873 26.760876 17.072281 23.925501 16.410806
## [281] 29.020678 17.292371 20.635299 25.728873 16.798215 20.636383 22.252292
## [288] 23.011414 34.451413 16.873577 29.344543 14.943718 22.154342 18.095111
## [295] 20.086679 32.044576 19.042483 22.853552 17.134832 24.745990 15.788748
## [302] 22.196745 36.378759 25.201378 18.321935 24.095309 22.984950 24.135402
## [309] 29.094054 28.826478 23.871794 15.035974 26.337323 28.136427 23.805393
## [316] 24.742890 16.033330 17.458203 23.318582 22.550071 32.073378 18.855533
## [323] 25.709672 22.254693 28.532292 19.508064 28.836492 20.763070 29.838354
## [330] 23.477935 24.626354 22.368233 20.822998 24.462529  8.374765 19.708865
## [337] 21.405013 23.487388 21.227952 30.820140 18.504549 11.291783 19.384716
## [344] 21.534960 23.726210 18.973502 10.863623 21.374417 28.564891 17.078760
```

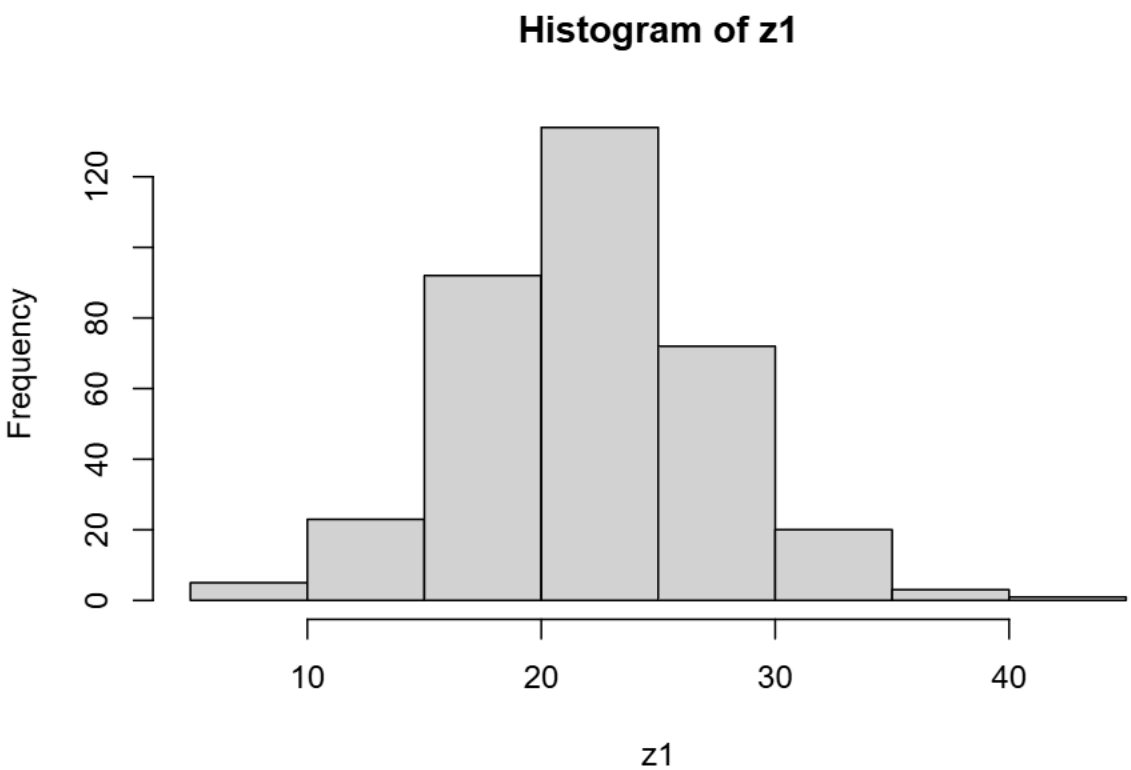
```r
w1<-length(z1)
w1
```

```
## [1] 350
```

```r
x1<-(501:850)
plot(z1,x1)
```




```r
hist(z1)
```

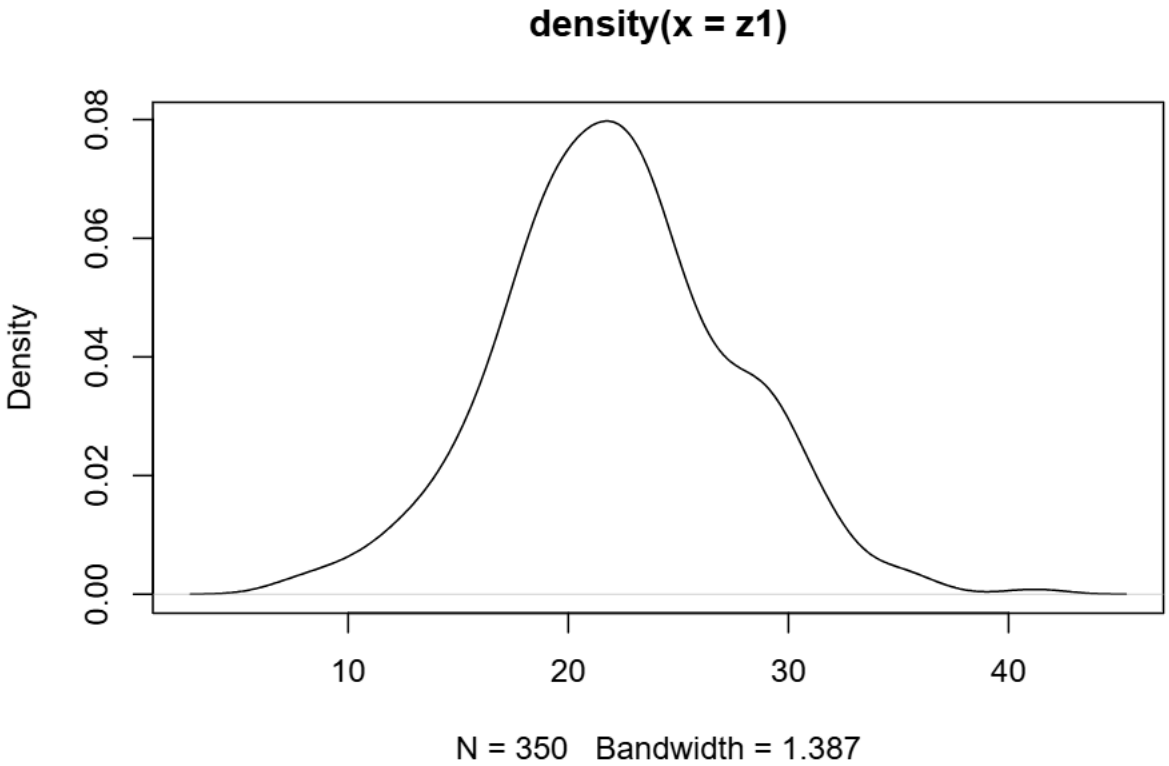



```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.387
```

```
##
##        x                y
##  Min.   : 2.835   Min.   :9.195e-06
##  1st Qu.:13.461   1st Qu.:9.587e-04
##  Median :24.087   Median :1.009e-02
##  Mean   :24.087   Mean   :2.348e-02
##  3rd Qu.:34.712   3rd Qu.:4.015e-02
##  Max.   :45.338   Max.   :7.976e-02
```

```
plot(density(z1))
```



Ejercicio 1 de algoritmos

Consignas, las ultimas 3 cifras de mi DNI son 214, crear una variable que tenga ese numero

```
DNI=214
```

Crear un vector que tenga todos los números enteris desde el 1 al 214.

```
secuencia_dni<-(1:214)
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
```

```
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214
```

Ejercicio 2

Calcular la suma de todos los valores del vector secuencia_dni

```r
total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  total <- total + i
}

total
```

```
## [1] 23005
```

Ejercicio 3

Repite el ejercicio anterior pero en Phyton

```python
total = 0
# Acá está la magia: le agregamos "r." para traer la variable desde R
valor_final = len(r.secuencia_dni)

for i in range(1, valor_final + 1):
    total = total + i

print(total)
```

```
## 23005
```

Ejercicio 4

¿Cuanto tarda en correr el código del ejercicio 2?

```r
inicio <-Sys.time()
total <- 0
valor_final <-1000000*length(secuencia_dni)

for (i in 1:valor_final) {
  total <- total + i
}

total
```

```
## [1] 2.2898e+16
```

```r
final<-Sys.time()
final-inicio
```

```
## Time difference of 3.888966 secs
```

Actividades de Elementos de Programación (Actividad 1.4 hasta 1.8)

Actividad 4 Generar un vector secuencia

El Objetivo de esta actividad es demostrar que R, a ser un lenguaje vectorizado, es mucho mas eficiente usando sus funciones nativas que usando los tipicos bucles de programación.

```r
# 1. Medimos el tiempo del bucle 'for'
tiempo_for <- system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})

cat("Tiempo de ejecución con bucle FOR:\n")
```

```
## Tiempo de ejecución con bucle FOR:
```

```r
print(tiempo_for)
```

```
##    user  system elapsed
##   0.018   0.000   0.018
```

```r
# -------------------------------------------------------

# 2. Medimos el tiempo de la función nativa de R ('seq')
# Generamos la misma secuencia de pares hasta 100.000
tiempo_seq <- system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})

cat("\nTiempo de ejecución con la función SEQ de R:\n")
```

```
##
## Tiempo de ejecución con la función SEQ de R:
```

```r
print(tiempo_seq)
```

```
##    user  system elapsed
##       0       0       0
```

En la consola vamos a ver tres valores: `user`, `system`, y `elapsed`. El que más nos importa es **`elapsed`** (el tiempo real que tardó). Vamps a notar que el tiempo del bucle `for` es considerablemente mayor. Esto pasa porque en el código del ejemplo (`A <- 0`), R tiene que redimensionar el tamaño del vector `A` 50.000 veces en la memoria, lo cual consume muchos recursos. El comando `seq`, por otro lado, crea el vector de una sola vez y su tiempo debería ser prácticamente cero.

Actividad 5: La sucesión de Fibonacci.

```r
# 1. Definimos cuántos términos queremos generar
n_terminos <- 20

# 2. Creamos un vector vacío para guardar los números
# La función numeric() crea un vector de ceros del tamaño indicado
fib <- numeric(n_terminos)
```

```r
# 3. Asignamos los dos primeros valores fijos de la serie (0 y 1)
fib[1] <- 0
fib[2] <- 1

# 4. Usamos un bucle for empezando desde la posición 3
for (i in 3:n_terminos) {
  # El término actual es la suma de los dos anteriores
  fib[i] <- fib[i-1] + fib[i-2]
}

# 5. Imprimimos el resultado
print(fib)
```

```
##  [1]    0    1    1    2    3    5    8   13   21   34   55   89  144  233  377
## [16]  610  987 1597 2584 4181
```

La lógica principal es que, salvo los dos primeros números (0 y 1), cada nuevo número se calcula sumando los dos inmediatamente anteriores. Para esto, en R (y en casi cualquier lenguaje), la herramienta ideal es el **bucle `for`**, ya que necesitamos que el programa "recuerde" los valores calculados en los pasos previos para calcular el siguiente.

Actividad 6:Definición matemática recurrente

Como ahora no sabemos de antemano cuántos números necesitamos generar (sino que dependemos de una condición: que el número sea mayor a 1.000.000), el bucle for ya no es la mejor opción. En su lugar, es ideal usar un bucle while (mientras). El bucle while se va a repetir indefinidamente mientras la condición que le pongamos siga siendo verdadera.

```r
# 1. Inicializamos los valores base de la serie
f_previo <- 0    # f_0
f_actual <- 1    # f_1

# 2. Iniciamos un contador de iteraciones
# Empezamos en 1 porque ya tenemos definido el primer término (f_1)
iteraciones <- 1

# 3. Bucle while: "Mientras el número actual sea menor o igual a 1.000.000"
while (f_actual <= 1000000) {

  # Calculamos el siguiente término sumando los dos anteriores
  siguiente <- f_actual + f_previo

  # Actualizamos las variables para prepararnos para la próxima vuelta
  f_previo <- f_actual
  f_actual <- siguiente

  # Le sumamos 1 al contador de iteraciones
  iteraciones <- iteraciones + 1
}

# 4. Imprimimos la respuesta a la consigna
cat("Iteraciones necesarias:", iteraciones, "\n")
```

```
## Iteraciones necesarias: 31
```

```r
cat("El primer número de la serie mayor a 1.000.000 es:", f_actual, "\n")
```

```
## El primer número de la serie mayor a 1.000.000 es: 1346269
```

Actividad 7: Ordenación de un vector por método burbuja

CONSIGNA: Compara la performance de ordenación del método burbuja vs el método sort de R.

```r
# 1. Instalar y cargar la librería (si no la tenés, sacale el # a la primera línea)
# install.packages("microbenchmark")
library(microbenchmark)

# 2. Definimos la función burbuja tal como está en tu apunte
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)){
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# 3. Creamos la muestra de tamaño 20.000
# Usamos set.seed para que los números aleatorios sean siempre los mismos
set.seed(123)
x_muestra <- sample(1:100000, 20000, replace = TRUE)

# 4. Comparamos la performance de ambos métodos
cat("Iniciando la prueba... (esto puede tardar unos minutos)\n")
```

```
## Iniciando la prueba... (esto puede tardar unos minutos)
```

```r
comparacion <- microbenchmark(
  Metodo_Burbuja = burbuja(x_muestra),
  Metodo_Sort_R = sort(x_muestra),
  times = 5 # Bajamos las repeticiones para que no tarde una eternidad
)

# 5. Mostramos los resultados finales
print(comparacion)
```

```
## Unit: microseconds
##            expr         min          lq         mean       median           uq
##  Metodo_Burbuja 22881323.28 22899826.49 2.412996e+07 23751393.351 24069226.998
##   Metodo_Sort_R      572.23      577.56 8.600026e+02      765.481     1122.841
##           max neval
##  27048015.844     5
##      1261.901     5
```

Actividad 8: La Penitencia de Newton

Consigna de trabajo : Desarrolla dos algoritmos que hagan este trabajo , por ejemplo para sumar desde 1 hasta 1x10^10 y verifica cuál de los dos es más eficiente.

```r
library(microbenchmark)

# 1. Definimos el límite (usamos 1 millón por seguridad de memoria)
n_limite <- 1000000

# Algoritmo 1: Fuerza bruta (El método del alumno castigado)
# Va sumando iterativamente número por número
suma_bucle <- function(n) {
  total <- 0
  for (i in 1:n) {
    total <- total + i
  }
  return(total)
}

# Algoritmo 2: El método analítico (La "avivada" matemática)
# Resuelve todo en un solo paso matemático
suma_formula <- function(n) {
  # Usamos as.numeric() para evitar desbordamiento de enteros con números gigantes
  n_num <- as.numeric(n)
  return((n_num * (n_num + 1)) / 2)
}

# 2. Comprobamos que ambos métodos den el mismo resultado
cat("Suma con Bucle For:", suma_bucle(n_limite), "\n")
```

```
## Suma con Bucle For: 500000500000
```

```r
cat("Suma con Fórmula Matemática:", suma_formula(n_limite), "\n\n")
```

```
## Suma con Fórmula Matemática: 500000500000
```

```r
# 3. Comparamos la eficiencia
cat("Comparando performance... \n")
```

```
## Comparando performance...
```

```r
comparacion_newton <- microbenchmark(
  Metodo_Bucle = suma_bucle(n_limite),
  Metodo_Formula = suma_formula(n_limite),
  times = 10
)

print(comparacion_newton)
```

```
## Unit: nanoseconds
##            expr      min       lq       mean     median       uq      max neval
##    Metodo_Bucle 15235482 15528883 19636810.9 17280154.0 25100160 29426644    10
##  Metodo_Formula      480      850   323891.2     4315.5    11620  3193662    10
```

2.0 Comparación de Performance con Gráficos de Violín

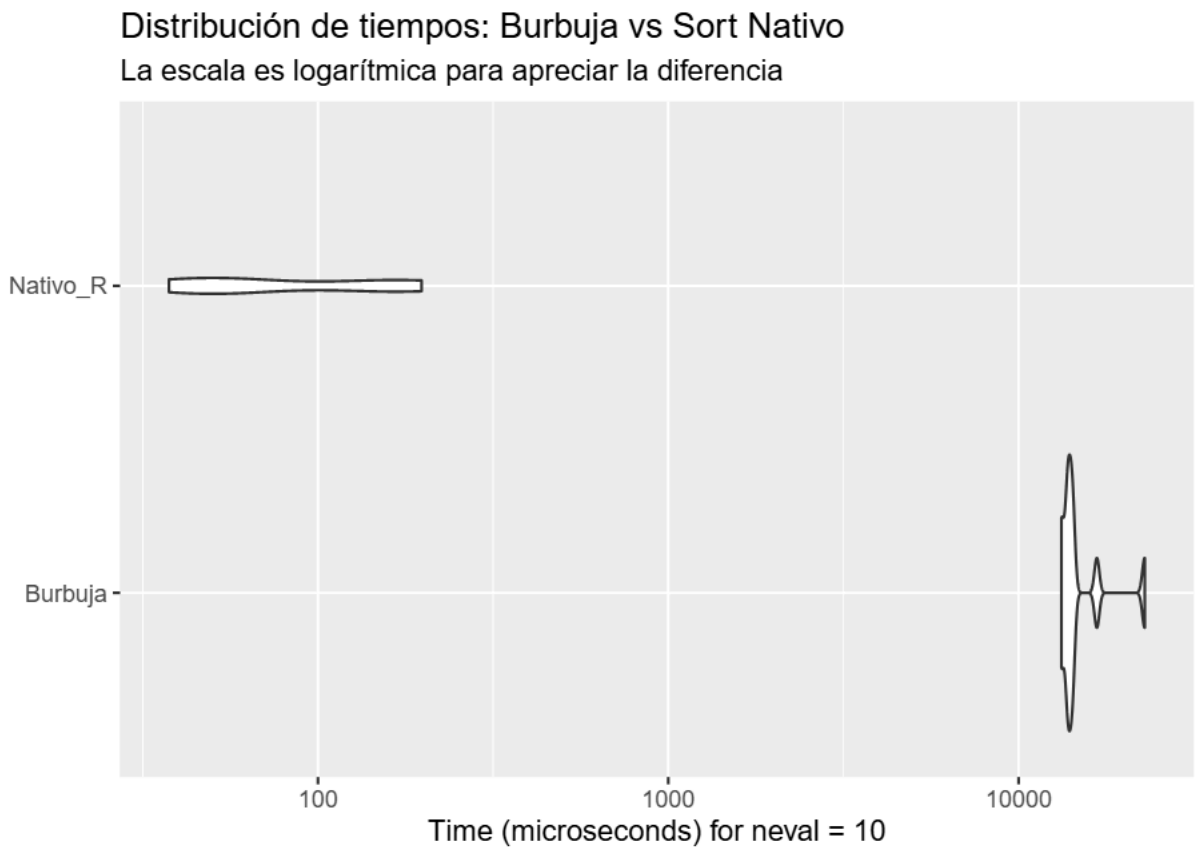
Instalamos y cargamos la librería necesaria

```r
if(!require(microbenchmark)) install.packages("microbenchmark")
if(!require(ggplot2)) install.packages("ggplot2")
```

```
## Loading required package: ggplot2
```

```r
library(microbenchmark)
library(ggplot2)

# 1. Definición de la función (Faltaba esto)
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n - 1)) {
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# 2. Comparación
set.seed(123)
vector_prueba <- sample(1:500, 500)

comparativa_sort <- microbenchmark(
  Burbuja = bubble_sort(vector_prueba),
  Nativo_R = sort(vector_prueba),
  times = 10
)

# 3. Graficado
autoplot(comparativa_sort) +
  labs(title = "Distribución de tiempos: Burbuja vs Sort Nativo",
       subtitle = "La escala es logarítmica para apreciar la diferencia",
       y = "Tiempo de ejecución", x = "Algoritmo")
```

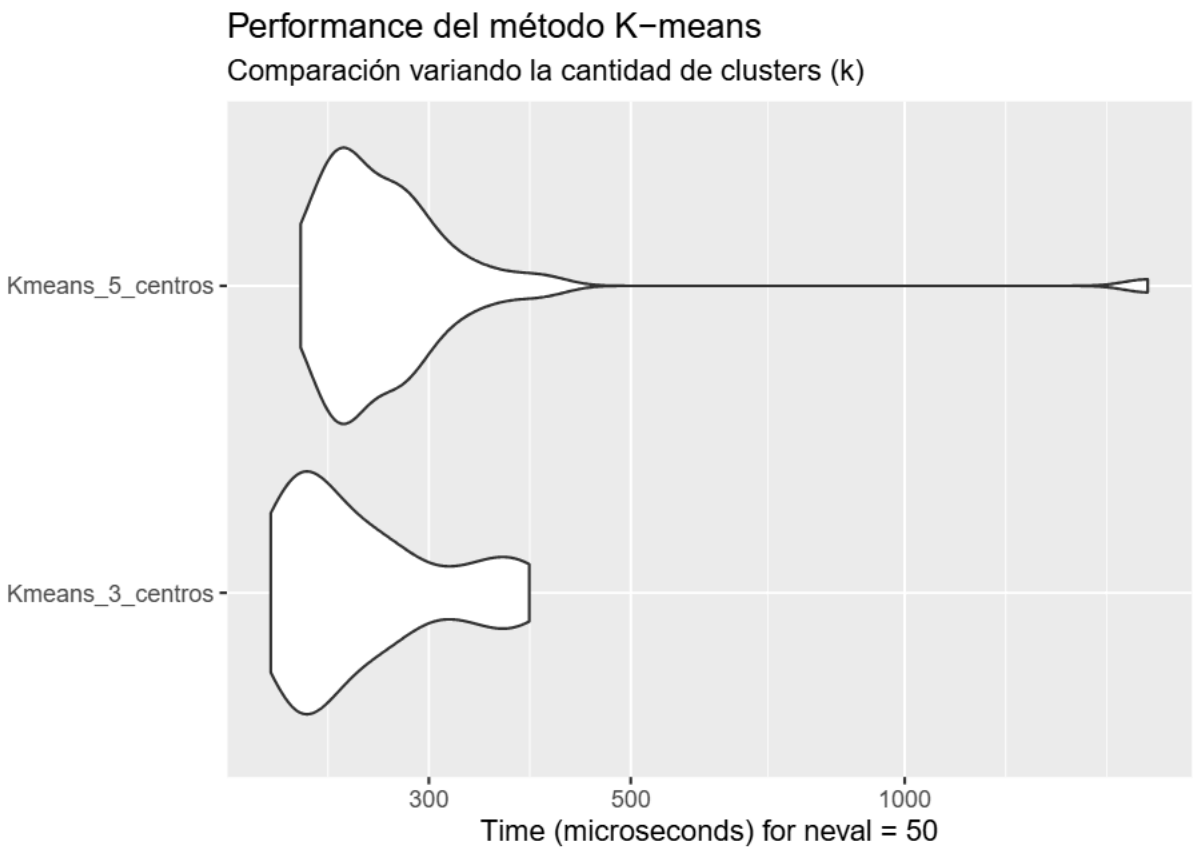
```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
##   Please report the issue at
##   <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Introducción Teórica a K-means:

El algoritmo de K-means es un método de agrupamiento (clustering) que particiona un conjunto de n observaciones en k grupos. Funciona de forma iterativa asignando cada punto al "centroide" más cercano y recalculando dichos centros hasta que las posiciones se estabilizan. Es ampliamente utilizado en minería de datos para encontrar patrones no visibles a simple vista.

```
# Medimos el rendimiento de kmeans con un dataset interno de R (iris)
comparativa_kmeans <- microbenchmark(
  Kmeans_3_centros = kmeans(iris[, 1:4], centers = 3),
  Kmeans_5_centros = kmeans(iris[, 1:4], centers = 5),
  times = 50
)

# Graficamos los resultados de K-means
autoplot(comparativa_kmeans) +
  labs(title = "Performance del método K-means",
       subtitle = "Comparación variando la cantidad de clusters (k)",
       y = "Tiempo de ejecución", x = "Configuración")
```


Performance del método K−means
Comparación variando la cantidad de clusters (k)
Kmeans_5_centros
Kmeans_3_centros
300
500
1000
Time (microseconds) for neval = 50